

# Track D: Finding errors in code

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

## Contents

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>Track D: Finding errors in code .....</b>            | <b>2</b> |
| D3 Silent errors .....                                  | 3        |
| D3-L3 Mutable defaults and the truthiness of zero ..... | 3        |
| Track D best-practice checklist.....                    | 8        |
| <b>Appendix: solutions and answer keys .....</b>        | <b>9</b> |

## Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

## D3 Silent errors

Roadmap: Every month. Join explosion, nulls in filters, type coercion, timezone drift, mutable defaults.

### D3-L3 Mutable defaults and the truthiness of zero

---

#### Concept

Two Python habits produce wrong results with no error message. Both appear in code written by people and by AI assistants, including your reference pipeline.

**Mutable default arguments.** Python evaluates a default value once, when it defines the function. Every call that relies on the default shares that one object. Write `def add_check(result, results=[])`, and each call appends to the same list. The second validation run returns the first run's checks as well.

The fix is a `None` sentinel:

```
1 def add_check(result: str, results: list[str] | None = None) -> list[str]:
2     results = [] if results is None else results
3     results.append(result)
4     return results
```

ruff flags the pattern as rule B006. Treat that warning as a bug.

**Truthiness of numbers.** `if value:` is `False` for `None`, and also for `0`, `0.0`, empty strings and empty containers. When zero is a valid measurement, such as a 0% reach, a zero weight or zero rows, a truthiness test treats it as missing.

Your reference `check_wave_stability` does this:

```
1 if current_pct and prior_pct:
```

A metric that falls from 12% to 0% skips the comparison. The largest possible drop produces no warning.

Test for the case you mean. For missing, write `is None`. For zero, write `== 0`.

```
1 if current_pct is not None and prior_pct is not None:
```

The same rule covers type hints. `error_message: str = None` claims the value is always a string. Write `str | None = None`, and mypy then makes you handle the `None` case.

## Why it matters

A questionnaire routing error removes one question for a whole wave. Its weighted percentage drops from 12% to 0.0%. The wave-over-wave stability check runs `if current_pct and prior_pct:`, treats 0.0 as missing and skips the metric. QA reports PASS. The dashboard publishes 0% reach for that question, and a client asks why the channel "lost its whole audience" in a week.

The check existed to catch exactly this change. One truthiness test disabled it for the one value that mattered most.

## Worked example: weak code

```
❷ def compare_waves(current: dict, prior: dict, tolerance: float = 0.10,
    flags=[]):
    2     """Flag metrics whose share moved more than tolerance between
        waves."""
    3     for metric, current_pct in current.items():
    4         prior_pct = prior.get(metric)
    ❸     if current_pct and prior_pct:
    6         if abs(current_pct - prior_pct) > tolerance:
    7             flags.append(metric)
    8     return flags
```

1. **Line 1.** Bare `dict` hides the value type. The values are floats or `None`.
2. **Line 1.** The default list is created once. Flags from one call leak into the next.
3. **Line 5.** A share of 0.0 is falsy, so the biggest possible drop is never compared.

## Worked example: strong code

```
1 def compare_waves(
    ❶     current: dict[str, float | None],
    3     prior: dict[str, float | None],
    4     tolerance: float = 0.10,
    5 ) -> list[str]:
    6     """Flag metrics whose share moved more than tolerance between waves.
    7
    8     A metric is skipped only when a share is missing. A share of 0.0 is
    9     compared.
    10    """
    ❷     flags: list[str] = []
    11     for metric, current_pct in current.items():
    12         prior_pct = prior.get(metric)
    ❸     if current_pct is None or prior_pct is None:
    14         continue
    15         if abs(current_pct - prior_pct) > tolerance:
    16             flags.append(metric)
    17     return flags
```

1. **Line 2.** The hint says a share can be missing, so mypy checks every use.

- 2. **Line 10.** A new list on every call. No state leaks between runs.
- 3. **Line 13.** Skips only missing values. 0.0 goes on to the comparison.

Both fixes are one line. Both bugs pass every test that uses only non-zero values and a single call.

## Recap card

- **When does Python evaluate a default argument?** Once, when it defines the function. A list or dict default is shared by every call.
- **Safe pattern for an optional list argument** Default to None. Inside: `results = [] if results is None else results.`
- **What is wrong with** `if current_pct and prior_pct: ?` 0.0 is falsy, so a real share of zero is treated as missing. Use `is not None`.
- **Type hint for an optional string that defaults to None** `str | None = None`. Never `str = None`.

## Practice

### Exercise 1. Predict the output (level 1)

What does this print?

```
1 def add_flag(flag, flags=[]):
2     flags.append(flag)
3     return flags
4
5 add_flag("null_weights")
6 print(add_flag("row_count"))
```

### Exercise 2. Multiple choice (level 2)

`current_pct = 0.0` and `prior_pct = 0.12`. The tolerance is 0.10. What does `check_wave_stability` from the reference code do with this metric?

- a) Flags it as WARN, because the change is 0.12.
- b) Marks it PASS.
- c) Leaves it out of the results.
- d) Raises a ZeroDivisionError.

### Why?

- r1) `if current_pct and prior_pct:` is False when `current_pct` is 0.0, so the block never runs.
- r2) The function filters out metrics below 1%.

### Exercise 3. Fill the gap (level 3)

Fix both signatures. The error message is optional, and the flags list must not be shared between calls.

```
1 def log_run(status: str, error_message: ____ (1) = None) -> None: ...
2
3 def collect(flag: str, flags: list[str] | None = ____ (2)) ->
  list[str]: ...
```

### Exercise 4. Spot the bug (level 3)

This check should warn when a share moves by more than the tolerance. A drop from 12% to 0% produces no warning. Tap the line with the bug, then fix the function.

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
  tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct and prior_pct:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Write down the line number of the bug. Then write the fix.

### Exercise 5. Write the function (level 4)

Write `compare_waves(current, prior, tolerance=0.10)`. Both arguments map metric names to a share or `None`. Return the metrics whose share changed by more than `tolerance`, in the order of `current`. Skip a metric only when either share is `None` or the metric is missing from `prior`. Calling the function twice must not carry results over.

Starter code:

```
1 def compare_waves(
2     current: dict[str, float | None],
3     prior: dict[str, float | None],
4     tolerance: float = 0.10,
5 ) -> list[str]:
6     ...
```

### Exercise 6. Refactor (level 5)

A validator collects check results with the helper below. The second contract validation in a job reports the first contract's failures too. Refactor `add_result` so each validation starts empty, keep the signature compatible for callers that pass a list, and add type hints and a docstring.

Starter code:

```
1 def add_result(name, passed, results=[]):  
2     results.append({"check": name, "passed": passed})  
3     return results
```

## Sources

- ruff rule B006 flags mutable default arguments. (verified: ruff 0.16.9 run in this repo reports B006 on `def f(x=[])`.)

## Track D best-practice checklist

- ☐ Read a traceback from the bottom up. Find the first frame in your own code.
- ☐ Reproduce the error on the smallest data that shows it.
- ☐ Check row counts before and after every join.
- ☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.
- ☐ Never use a mutable object as a default argument.
- ☐ Check every function and parameter an AI suggests against the official docs for your version.
- ☐ Run AI code on a small input with a known answer before you trust it.
- ☐ Write the failing test first. A test that only repeats the code proves nothing.
- ☐ Ask the model to list its assumptions. Check each one.
- ☐ State versions, constraints, expected outputs and tests in every prompt.

# Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

## D3-L3 Mutable defaults and the truthiness of zero

---

### D3-L3 exercise 1

**Hints.** (1) When is the default `[]` created? (2) Both calls use the default. Is it the same list? (3) The list is created once, when Python defines the function.

**Answer.**

```
1 ['null_weights', 'row_count']
```

The default list is created when Python defines the function. Both calls append to that one list, so the second call returns both flags.

**Common wrong answers.**

- `['row_count']`: That is what the author intended. The default list is created once, so the second call appends to the list the first call used.
- `['row_count', 'null_weights']`: `append` adds to the end. The first call's flag is already in the list.

### D3-L3 exercise 2

**Hints.** (1) Evaluate `0.0` and `0.12` in your head. (2) `0.0` is falsy, like `None`. (3) The whole block, including `results.append`, is skipped.

**Answer.** c) Leaves it out of the results. Because: r1) `if current_pct and prior_pct:` is False when `current_pct` is 0.0, so the block never runs.

`0.0` and `0.12` evaluates to `0.0`, which is falsy. The block that compares and appends never runs, so the metric vanishes from the results and no warning appears.

**Common wrong answers.**

- a: That is what the check should do. Look at the `if` line before the comparison: `0.0` is falsy.
- b: Nothing is appended for this metric, so it cannot be marked PASS. The `if` skips it.
- d: The check subtracts, it does not divide. No error appears, which is the problem.

- `c|r2`: Right outcome, wrong reason. There is no 1% filter. The truthiness test treats 0.0 as missing.

### D3-L3 exercise 3

**Hints.** (1) The first default is `None`, so the type must allow `None`. (2) The second default must be immutable. (3) `str | None` for the first gap and `None` for the second.

**Answer.**

```
1 def log_run(status: str, error_message: str | None = None) -> None: ...
2
3 def collect(flag: str, flags: list[str] | None = None) -> list[str]: ...
```

`error_message: str | None = None` states that the value may be missing.

`flags: list[str] | None = None` avoids a shared list. Create the list inside the function when it is `None`.

**Common wrong answers.**

- `g1 = str: str = None` claims the value is always a string. mypy in strict mode rejects it. Write `str | None`.
- `g2 = []`: A list default is created once and shared by every call. Use `None` and create the list inside the function.

### D3-L3 exercise 4

**Hints.** (1) Which values make `if current_pct` and `prior_pct`: `False`? (2) 0.0 is one of them. (3) Replace the truthiness test with `is not None` on both values.

**Answer.** The bug is on line 3.

Fixed version:

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
2                       tolerance: float = 0.10) -> str:
3     """Return WARN, PASS, or SKIP when a share is missing."""
4     if current_pct is not None and prior_pct is not None:
5         change = abs(current_pct - prior_pct)
6         return "WARN" if change > tolerance else "PASS"
7     return "SKIP"
```

Line 3 treats 0.0 as missing. Use `if current_pct is not None and prior_pct is not None` so that only missing shares skip the comparison.

**Common wrong answers.**

- Line 4: The subtraction is correct. The code never reaches it when a share is 0.0.

- Line 5: The comparison with the tolerance is correct. Look at what stops the code reaching it.

### D3-L3 exercise 5

**Hints.** (1) Start with `flags: list[str] = []` inside the function. (2) Read the prior share with `prior.get(metric)`, which gives `None` for a missing metric. (3) Skip with `if current_pct is None or prior_pct is None: continue`, then compare with the tolerance.

**Answer.**

```

1 def compare_waves(
2     current: dict[str, float | None],
3     prior: dict[str, float | None],
4     tolerance: float = 0.10,
5 ) -> list[str]:
6     """Flag metrics whose share moved more than tolerance between waves.
7
8     A metric is skipped only when a share is missing. A share of 0.0 is
9     compared.
10    """
11     flags: list[str] = []
12     for metric, current_pct in current.items():
13         prior_pct = prior.get(metric)
14         if current_pct is None or prior_pct is None:
15             continue
16         if abs(current_pct - prior_pct) > tolerance:
17             flags.append(metric)
18     return flags

```

Create the list inside the function. Use `prior.get` so a new metric gives `None`. Skip only on `None`, then compare the absolute change with the tolerance.

### D3-L3 exercise 6

**Hints.** (1) Default `results` to `None`. (2) Create a new list only when `results` is `None`. (3) `results or []` looks similar but replaces an empty list the caller passed. Use `is None`.

**Answer.**

```

1 def add_result(
2     name: str, passed: bool, results: list[dict[str, object]] | None =
3     None
4 ) -> list[dict[str, object]]:
5     """Append one check result and return the list.
6
7     Args:
8         name: Check name.
9         passed: Whether the check passed.
10        results: List to append to. A new list is created when omitted.

```

```
10
11     Returns:
12         The list with the new result.
13     """
14     results = [] if results is None else results
15     results.append({"check": name, "passed": passed})
16     return results
```

Default to `None`, create the list when `results is None`, then append and return. Avoid `results or []`: an empty list the caller passed is falsy, so the function would append to a new list and the caller would never see the result.